Department of Electronics and Electrical Communication Engineering
Indian Institute of Technology Kharagpur

Embedded Systems Laboratory

EC39202

Experiment 1: Sorting, Searching, Polynomial Evaluation, and Substring Matching on 8051



Soumadipta Majumder
23EC30055

Jai Raj
23EC10031

Group 5

8 February, 2026

1 Problem Statement

Objective:

- **Experiment 01 (A):** Sort an array of 10 bytes stored in external memory from location 9000H onwards in ascending order. Then perform binary search for the number stored at location 9500H in the sorted array. If the number is present, store the *index* (not address) at location 9550H; otherwise store -1 at location 9550H.
- **Experiment 01 (B):** Given n at location 9500H, x at 9501H, and polynomial coefficients starting from 9502H, evaluate the polynomial

  $$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

  (equivalently using the nested form / Horner's method)

  $$a_0 + x\big(a_1 + x\big(a_2 + \cdots + x(a_{n-1} + x a_n)\cdots\big)\big),$$

  and store the result at location 9550H.
- **Experiment 01 (C):** Check whether a NULL-terminated string stored from location 9000H is a substring of another NULL-terminated string stored from location 9200H. If a substring is detected, store the start index (not address) at memory location 9500H; otherwise store -1 at 9500H.

Hardware/Software Used:

- Microcontroller: Intel 80C51BH
- Softwares Used: Keil μVision, Win51

2 Flow Chart

Experiment 01 (A) — SORT (Bubble Sort in xRAM)

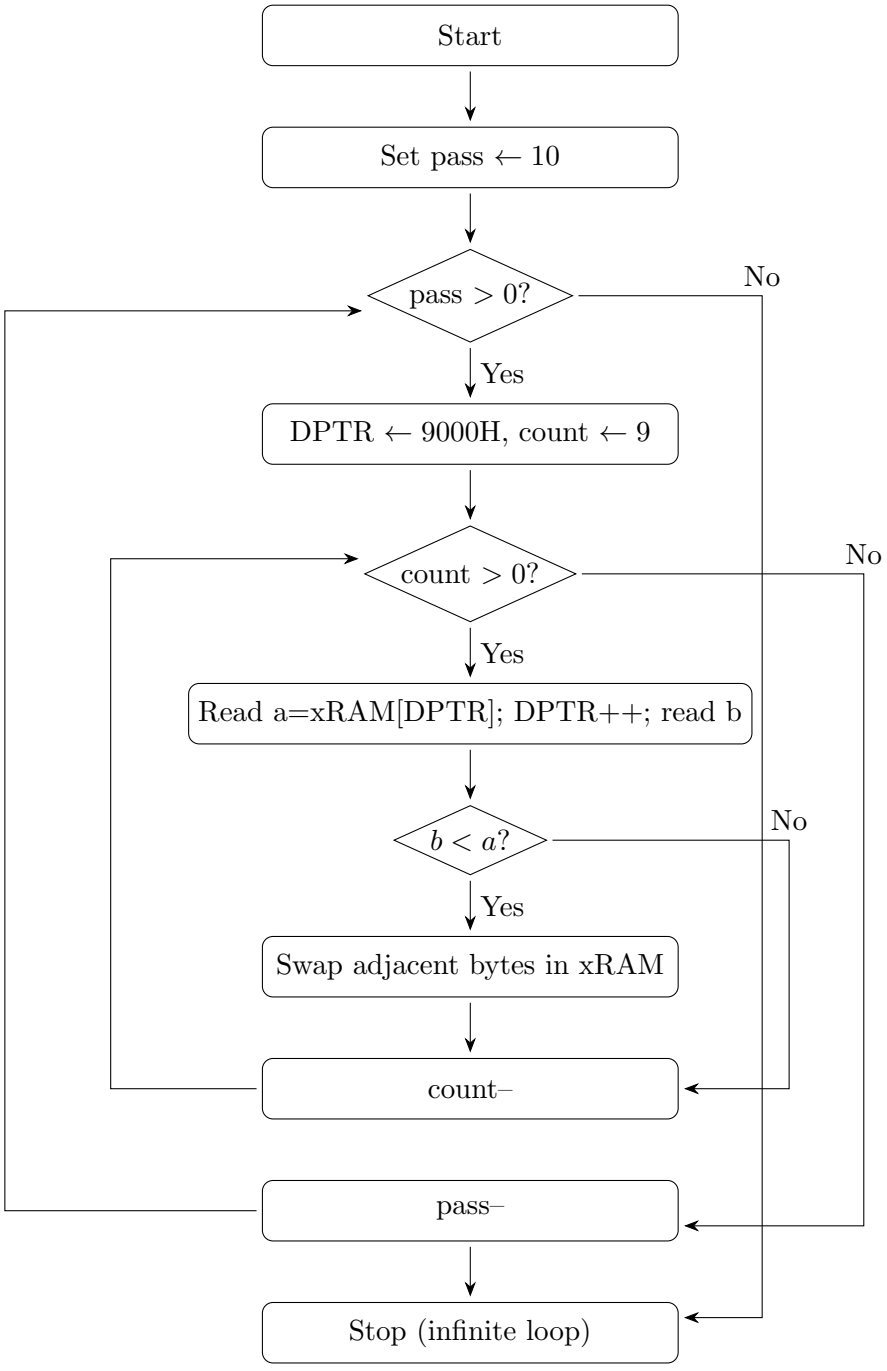


Figure 1: Flow chart for bubble sort of 10 bytes at xRAM 9000H–9009H.

Experiment 01 (A) — SEARCH (Binary Search in xRAM)

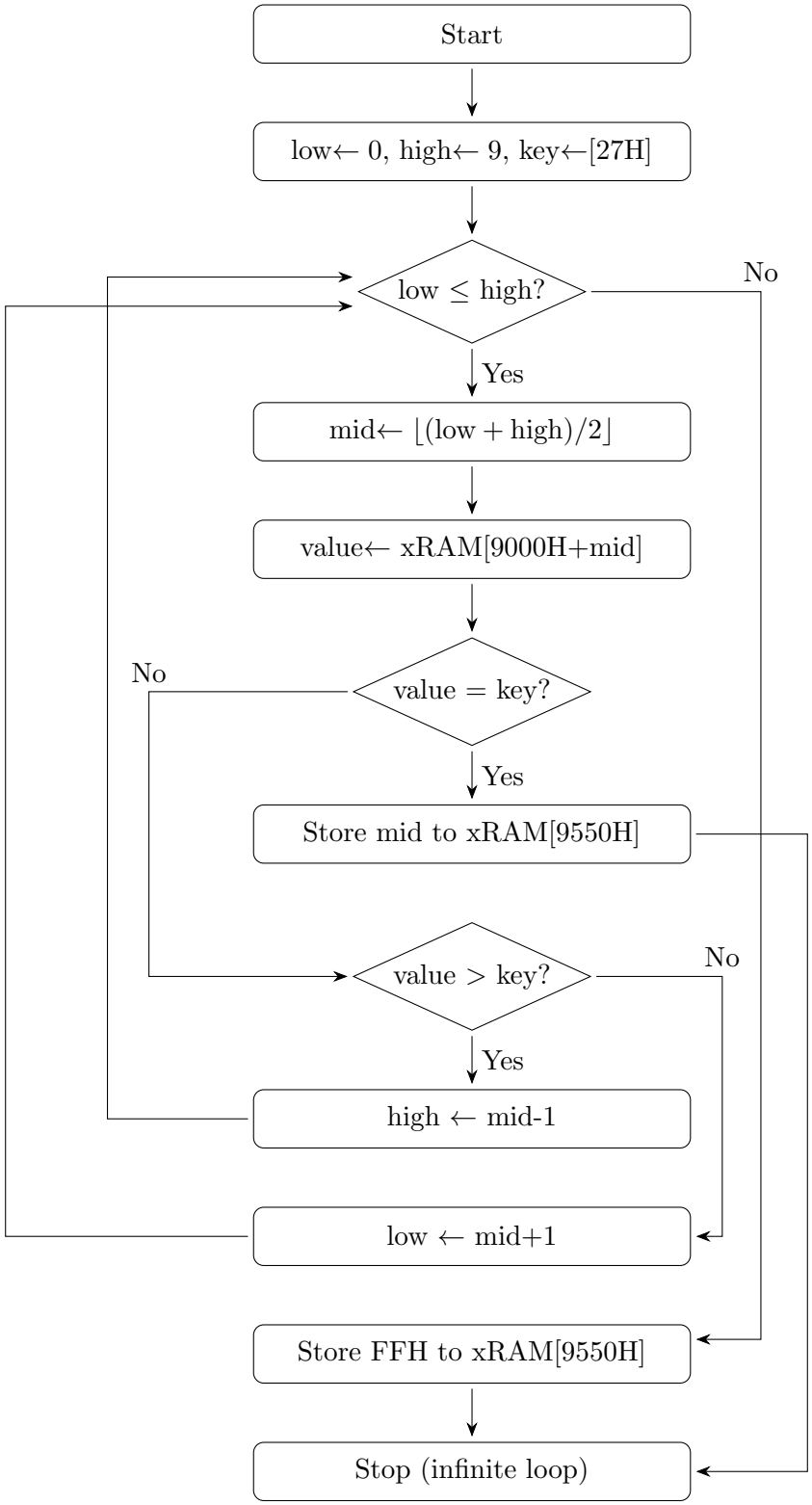


Figure 2: Flow chart for binary search in sorted array at xRAM 9000H–9009H.

Experiment 01 (B) — Polynomial / Iterative Accumulation

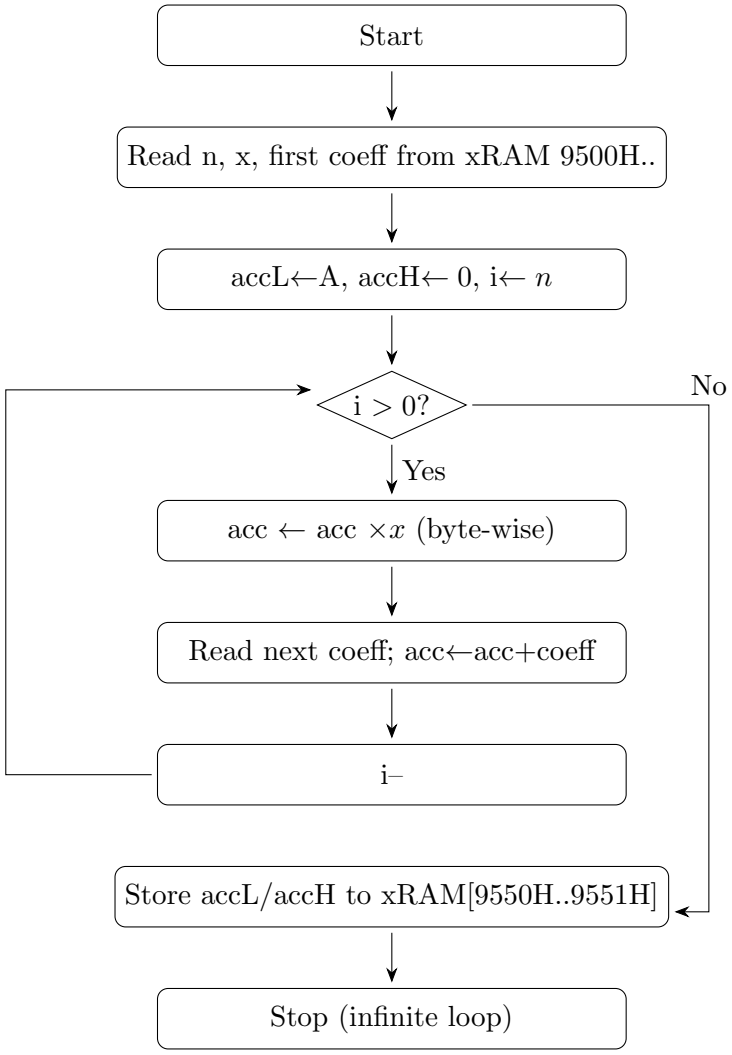


Figure 3: Flow chart for the coefficient-accumulation routine (Experiment 01B).

Experiment 01 (C) — Substring Search in xRAM

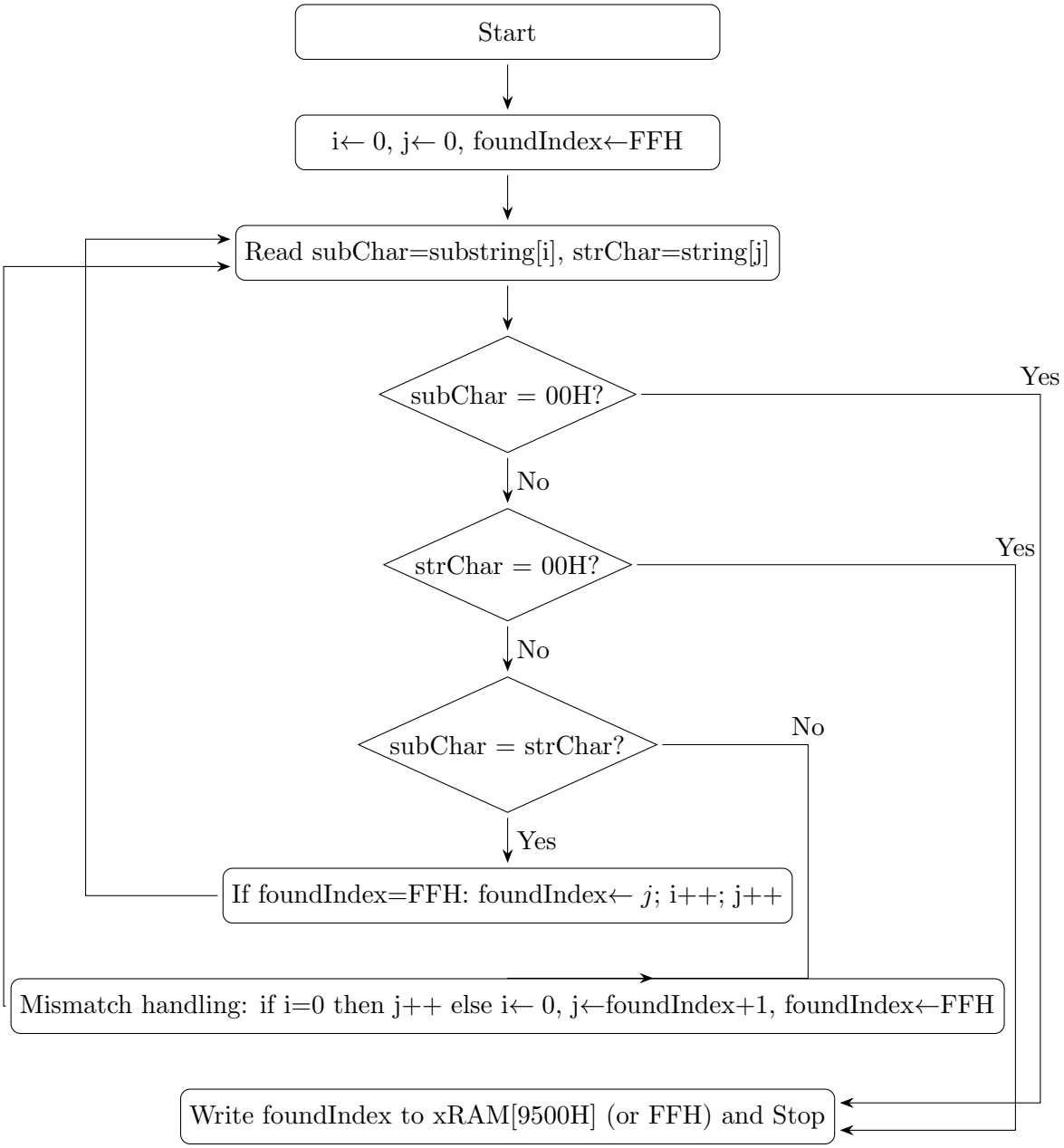


Figure 4: Flow chart for substring search (Experiment 01C).

3 Pseudo-Code / Algorithm

Experiment 01 (A) — SORT (Bubble Sort in xRAM)

1. Set `pass` $\leftarrow 10$.
2. Repeat while `pass` > 0:
   (a) Set `DPTR` $\leftarrow$ 9000H (base address of array).
   (b) Set `count` $\leftarrow 9$ (number of adjacent comparisons).
   (c) Repeat while `count` > 0:
      i. Read `a` $\leftarrow$ xRAM[DPTR].
      ii. Increment DPTR.
      iii. Read `b` $\leftarrow$ xRAM[DPTR].
      iv. If $b < a$ then swap the two bytes in xRAM so that a comes before b.
      v. Decrement `count`.
   (d) Decrement `pass`.
3. Stop (infinite loop).

Experiment 01 (A) — SEARCH (Binary Search in xRAM)

1. Initialize `low` $\leftarrow 0$, `high` $\leftarrow 9$.
2. Load `key` from internal RAM location 27H.
3. While `low` $\leq$ `high`:
   (a) `mid` $\leftarrow \lfloor$(`low` + `high`)$/2 \rfloor$.
   (b) Read `value` $\leftarrow$ xRAM[9000H + `mid`].
   (c) If `value` = `key`: store `mid` at xRAM[9550H] and stop.
   (d) Else if `value` $>$ `key`: set `high` $\leftarrow$ `mid` $- 1$.
   (e) Else: set `low` $\leftarrow$ `mid` $+ 1$.
4. If not found: store FFH at xRAM[9550H] and stop.

Experiment 01 (B) — Polynomial / Iterative Accumulation

1. Load `n` from xRAM[9500H], `x` from xRAM[9501H].
2. Load initial accumulator low byte `accL` from xRAM[9502H] (first coefficient) and set `accH` $\leftarrow 0$.
3. For $i = 1$ to n:
   (a) Compute `acc` $\leftarrow$ `acc` $\times x$ (byte-wise multiply and carry accumulation as in code).

(b) Read next coefficient `c` from xRAM and add it to `accL`; propagate carry into `accH`.

4. Store result: xRAM[9550H] ← `accL`, xRAM[9551H] ← `accH`.
5. Stop (infinite loop).

Experiment 01 (C) — Substring Search in xRAM

1. Initialize indices: `i` (substring index) ← 0, `j` (string index) ← 0.
2. Set base pages: substring in 90xxH, string in 92xxH.
3. Set `foundIndex` ← FFH (meaning "not yet found").
4. Repeat:
   (a) Read `subChar` ← substring[i].
   (b) Read `strChar` ← string[j].
   (c) If `subChar` is 00H: match complete ⇒ write `foundIndex` to xRAM[9500H] and stop.
   (d) If `strChar` is 00H: string ended ⇒ write FFH to xRAM[9500H] and stop.
   (e) If `subChar` = `strChar`:
      i. If `foundIndex` is FFH, set `foundIndex` ← j.
      ii. Increment `i` and `j` and continue.
   (f) Else (mismatch):
      i. If `i` = 0, increment `j` and continue.
      ii. Else reset `i` ← 0, set `j` ← (`foundIndex` + 1), and reset `foundIndex` ← FFH.

4 Actual Code (8051 Assembly) with Comments

Listing 1: Experiment 01 (A)-SORT — 8051 Assembly

```
ORG 8100H

START:
       MOV R1 , #0AH ; number of passes = 10

NEXT_PASS:
       MOV DPTR , #9000H ; start of array in xRAM
       MOV R0 , #09H ; 9 adjacent comparisons per pass (10 elements)

LOOP:
       ; Read first element into B
       MOVX A , @DPTR
       MOV B , A

       ; Move to next element and read it into A
       INC DPTR
```

```
        MOVX A , @DPTR

        ; Compare A (next) with B (current)
        ; If equal, no swap
        CJNE A , B , NEQ
        SJMP NO_SWAP

NEQ:
        ; After CJNE, Carry=1 if A < B.
        ; If Carry=0 (A > B), go to NO_SWAP.
        ; If Carry=1 (A < B), swap needed to keep ascending order.
        JNC NO_SWAP

        ; Swap: write B into current DPTR (second position)
        MOV R2 , A ; save A (second element)
        MOV A , B ; A = first element
        MOVX @DPTR , A ; write first element into second position

        ; Write saved A into previous address (first position)
        MOV A , R2
        DEC DPL ; DPTR = DPTR - 1 (back to first position)
        MOVX @DPTR , A
        INC DPTR ; DPTR back to second position for next step

NO_SWAP:
        DJNZ R0 , LOOP ; next adjacent pair in this pass
        DJNZ R1 , NEXT_PASS ; next pass

HERE:
        SJMP HERE ; stop (infinite loop)

END
```

Listing 2: Experiment 01 (A)-SEARCH — 8051 Assembly

```
ORG 8100H

; Initialize low/high and load key
MOV R0, #00H ; low = 0
MOV R1, #09H ; high = 9
MOV B, 27H ; key from internal RAM 27H

SEARCH_LOOP:
    ; If (high - low) < 0 then low > high => not found
    MOV A, R1
    CLR C
    SUBB A, R0
    JC NOT_FOUND

    ; mid = (low + high) / 2
    MOV A, R0
    ADD A, R1
    CLR C
    RR A ; rotate right through carry => divide by 2 (carry cleared)
```

```
    MOV R2, A ; R2 = mid

    ; Load value = xRAM[9000H + mid]
    MOV DPL, A
    MOV DPH, #90H
    MOVX A, @DPTR

    ; Compare value (A) with key (B)
    CJNE A, B, CHECK_SIDE
    SJMP FOUND

CHECK_SIDE:
    ; After CJNE, Carry=1 if A < B (value < key)
    JC VALUE_LESS

VALUE_GREATER:
    ; value > key => search left half: high = mid - 1
    MOV A, R2
    DEC A
    MOV R1, A
    SJMP SEARCH_LOOP

VALUE_LESS:
    ; value < key => search right half: low = mid + 1
    MOV A, R2
    INC A
    MOV R0, A
    SJMP SEARCH_LOOP

FOUND:
    ; Store found index (mid) to xRAM[9550H]
    MOV DPTR, #9550H
    MOV A, R2
    MOVX @DPTR, A
    SJMP FINAL

NOT_FOUND:
    ; Store FFH to xRAM[9550H]
    MOV DPTR, #9550H
    MOV A, #0FFH
    MOVX @DPTR, A

FINAL:
    ; Stop (infinite loop)
    SJMP FINAL

END
```

Listing 3: Experiment 01 (B) — 8051 Assembly

```
ORG 8100H

START:
       ; Point to start of input data
```

```
        MOV DPTR , #9500H

        ; Read n
        MOVX A, @DPTR
        MOV R0 , A ; R0 = n

        ; Read x
        INC DPTR
        MOVX A , @DPTR
        MOV R1 , A ; R1 = x

        ; Read first coefficient A_n (initialize accumulator)
        INC DPTR
        MOVX A , @DPTR
        MOV R3 , A ; acc low = A_n
        MOV R2 , #00H ; acc high = 0

LOOP:
        ; Compute (acc low) * x
        MOV A , R1 ; A = x
        MOV B , A
        MOV A , R3 ; A = acc low
        MUL AB ; AB = accLow * x
        MOV R3 , A ; new acc low = product low
        MOV R4 , B ; save product high for later add

        ; Compute (acc high) * x and add saved high from low multiply
        MOV A , R2 ; A = acc high
        MOV B , R1 ; B = x
        MUL AB ; AB = accHigh * x
        ADD A , R4 ; add carry-in from (accLow*x) high byte
        MOV R2 , A ; new acc high (low 8 bits)

        ; Load next coefficient and add to 16-bit accumulator
        INC DPTR
        MOVX A , @DPTR ; A = next coefficient
        ADD A , R3
        MOV R3 , A ; store updated low byte
        MOV A , R2
        ADDC A , #00H ; propagate carry into high byte
        MOV R2 , A

        DJNZ R0 , LOOP ; repeat n times

LAST:
        ; Store 16-bit result to xRAM[9550H..9551H]
        MOV DPTR , #9550H
        MOV A , R3
        MOVX @DPTR , A ; result low
        INC DPTR
        MOV A , R2
        MOVX @DPTR , A ; result high

HERE:
```

```
        ; Stop (infinite loop)
        SJMP HERE

END
```

Listing 4: Experiment 01 (C) — 8051 Assembly

```
ORG 8100H

; Initialize indices and base pages
MOV R0 , #00H ; i = 0 (substring index)
MOV R1 , #00H ; j = 0 (string index)
MOV R2 , #90H ; substring base page = 90xxH
MOV R3 , #92H ; string base page = 92xxH
MOV R7 , #0FFH ; foundIndex = FFH (not set yet)

LOOP:
        ; Load substring[i] into R4
        MOV DPH , R2
        MOV DPL , R0
        MOVX A , @DPTR
        MOV R4 , A

        ; Load string[j] into R5
        MOV DPH , R3
        MOV DPL , R1
        MOVX A , @DPTR
        MOV R5 , A

        ; If substring char is 00H => substring fully matched => FOUND
        CJNE R4 , #00H , RUNNING
        SJMP FOUND

RUNNING:
        ; If string char is 00H => string ended before match => NOT_FOUND
        CJNE R5 , #00H , STILL_RUNNING
        SJMP NOT_FOUND

STILL_RUNNING:
        ; Compare substring char and string char
        MOV A , R4
        MOV B , R5
        CJNE A , B , CHECKING ; mismatch => handle restart logic
        SJMP E_Q ; equal => advance indices

E_Q:
        ; On first equal of a match attempt, capture start index (j)
        MOV A , R7
        CJNE A , #0FFH , SKIP_SET_START
        MOV A , R1
        MOV R7 , A ; foundIndex = j

SKIP_SET_START:
        ; Advance i and j to continue matching
```

```
        INC R0
        INC R1
        SJMP LOOP

CHECKING:
        ; Mismatch handling
        ; Save (foundIndex + 1) in R6 so we can restart matching from next position
        MOV A , R7
        INC A
        MOV R6 , A
        MOV R7 , #0FFH ; clear foundIndex (no active match)

        ; If mismatch happened when i == 0, just move to next character in string (j++)
        CJNE R0 , #00H , ANOTHER_CHECK
        INC R1
        SJMP LOOP

ANOTHER_CHECK:
        ; If mismatch happened after some matches, restart:
        ; i = 0, j = foundIndex + 1
        MOV R0 , #00H
        MOV A , R6
        MOV R1 , A
        SJMP LOOP

FOUND:
        ; Write foundIndex to xRAM[9500H]
        MOV DPTR , #9500H
        MOV A , R7
        MOVX @DPTR , A
        SJMP HERE

NOT_FOUND:
        ; Write FFH to xRAM[9500H]
        MOV DPTR , #9500H
        MOV A , #0FFH
        MOVX @DPTR , A
        SJMP HERE

HERE:
        ; Stop (infinite loop)
        SJMP HERE

END
```

5 Results

Note: Include screenshots (and a short video) showing the number mentioned on the PC number.

Experiment 01 (A) — SORT (Bubble Sort)

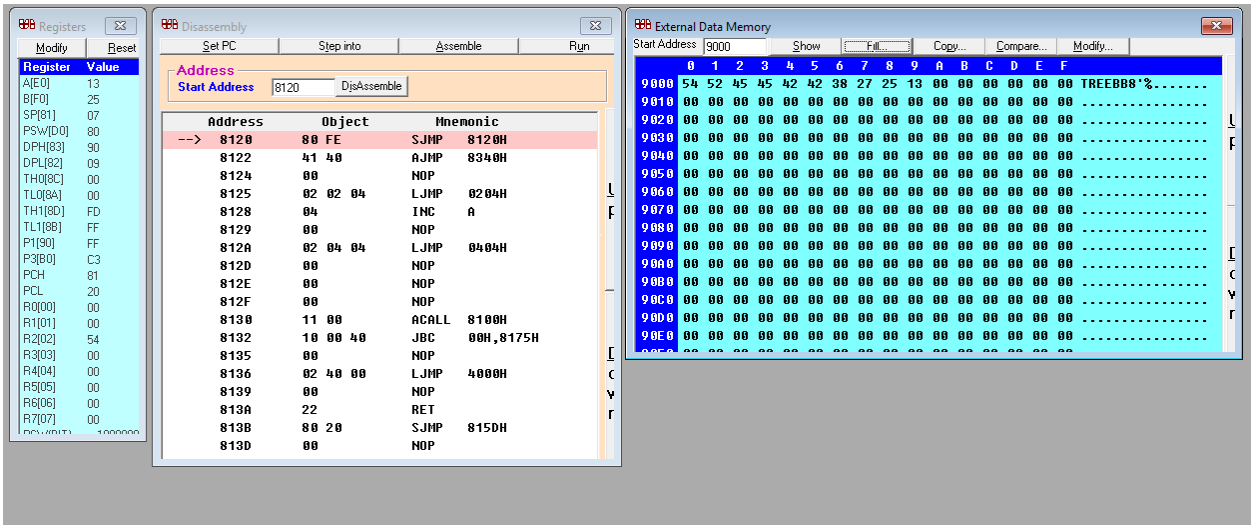


Figure 5: Experiment 01(A) SORT: Output screenshot 1.

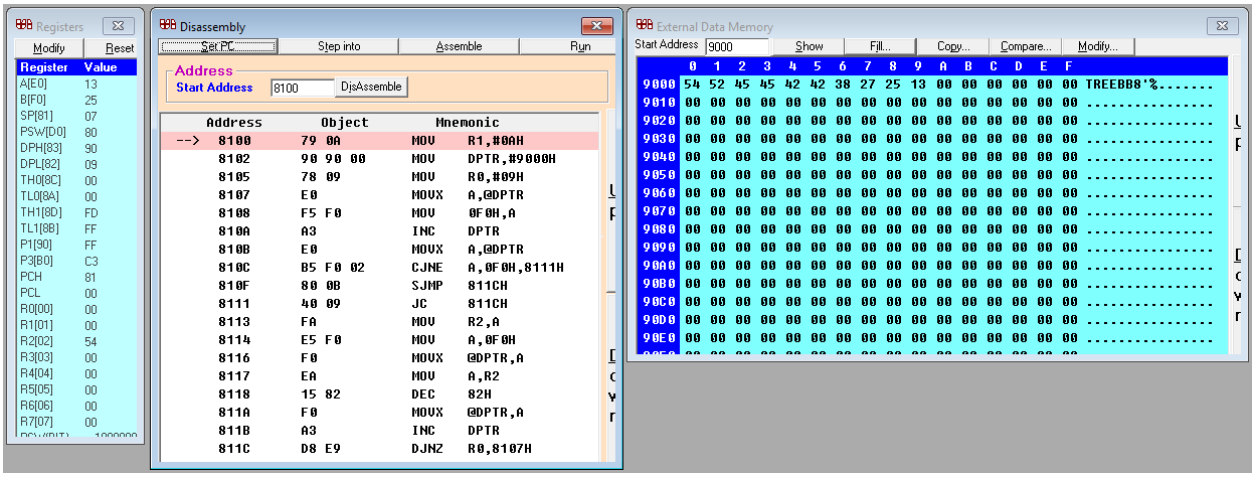


Figure 6: Experiment 01(A) SORT: Output screenshot 2.

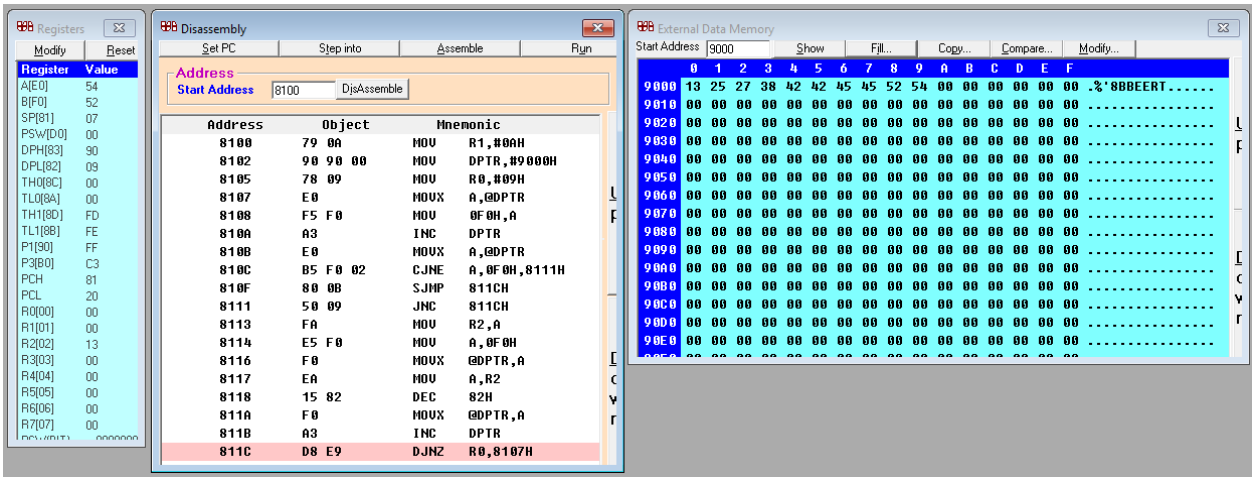


Figure 7: Experiment 01(A) SORT: Output screenshot 4.

Experiment 01 (A) — SEARCH (Binary Search)

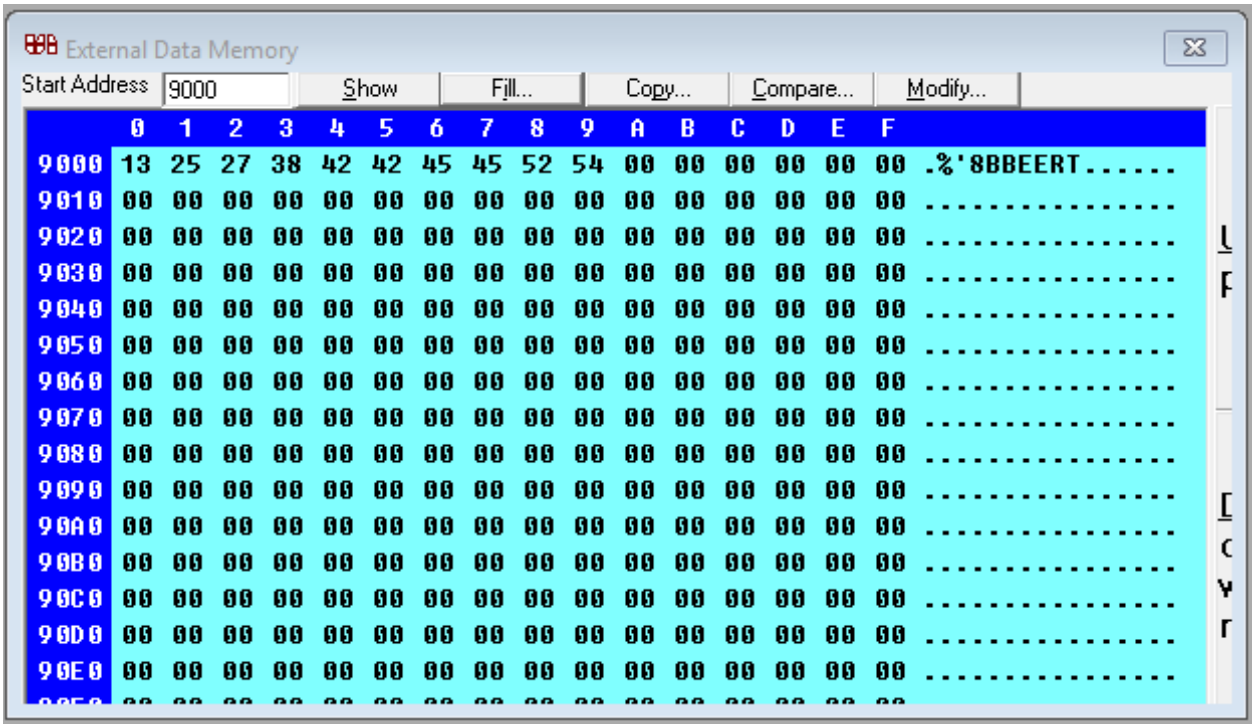


Figure 8: Experiment 01(A) SEARCH: Output screenshot 1.

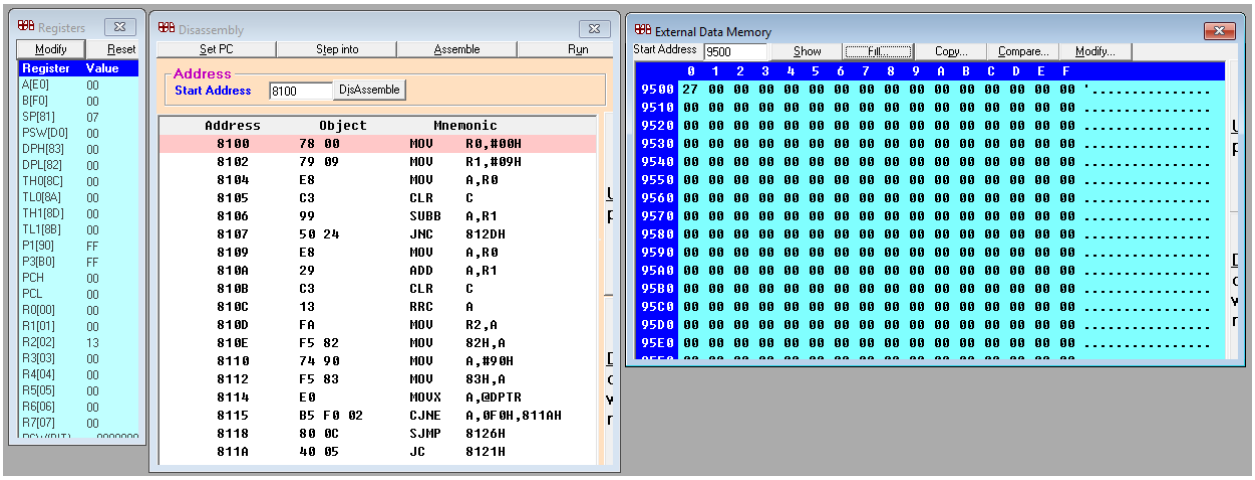


Figure 9: Experiment 01(A) SEARCH: Output screenshot 2.

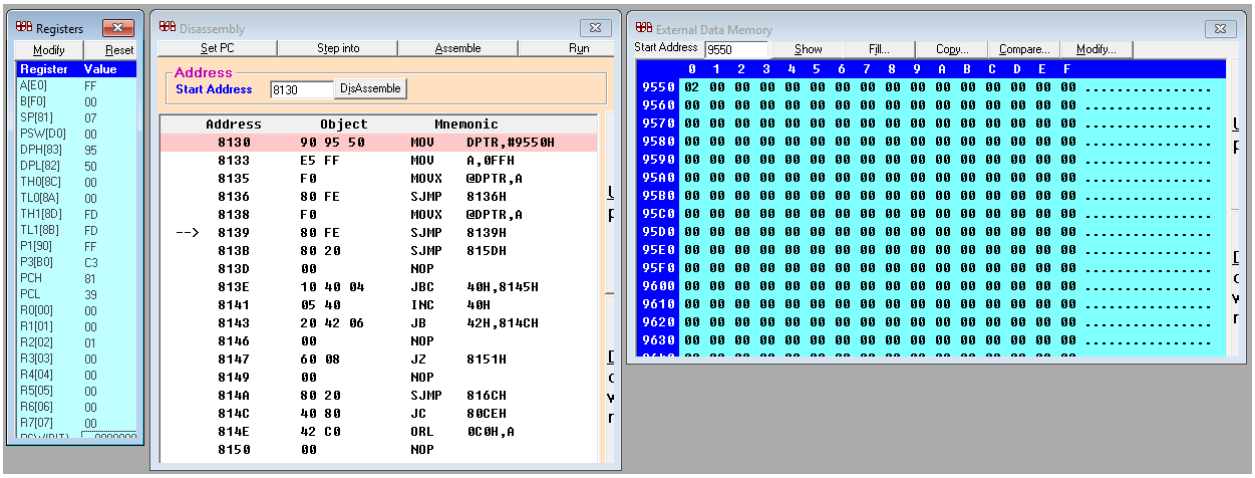


Figure 10: Experiment 01(A) SEARCH: Output screenshot 3.

Experiment 01 (B)

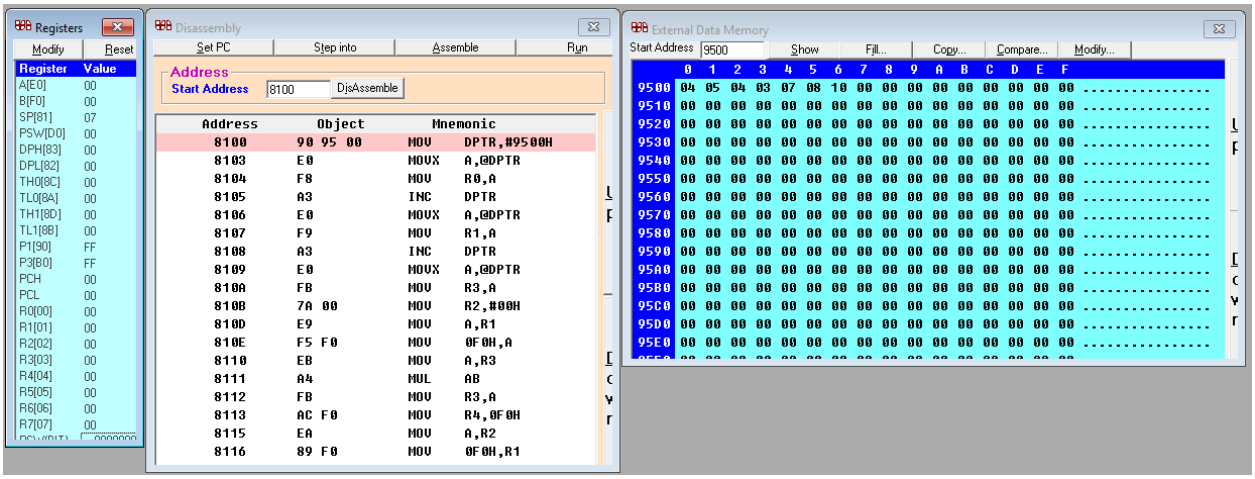


Figure 11: Experiment 01(B): Output screenshot 2.

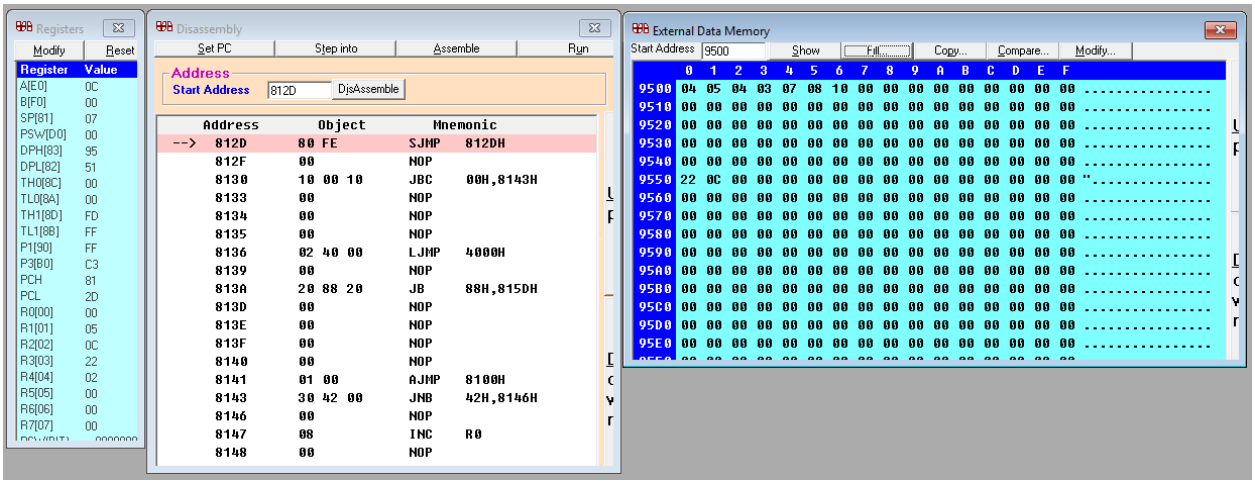


Figure 12: Experiment 01(B): Output screenshot 3.

Experiment 01 (C)

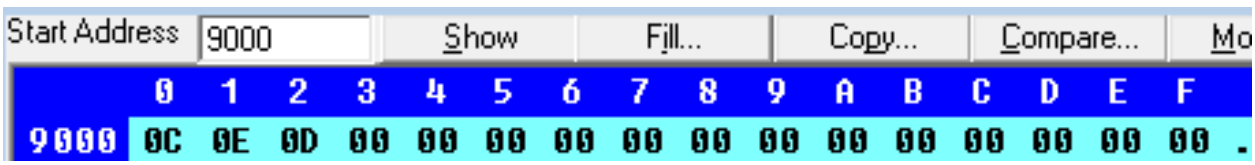


Figure 13: Experiment 01(C): Output screenshot 1.

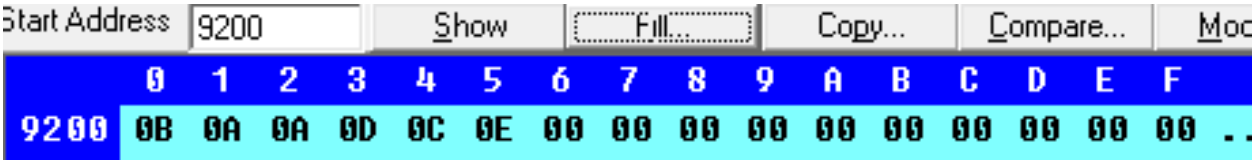


Figure 14: Experiment 01(C): Output screenshot 2.

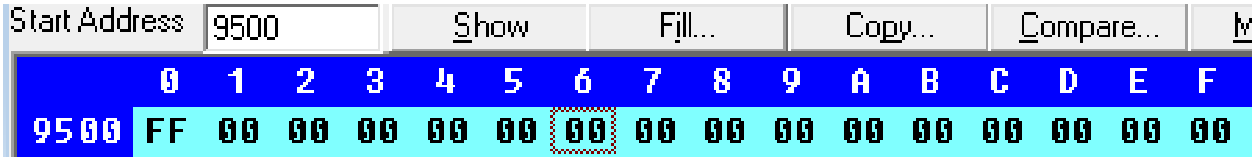


Figure 15: Experiment 01(C): Output screenshot 3.

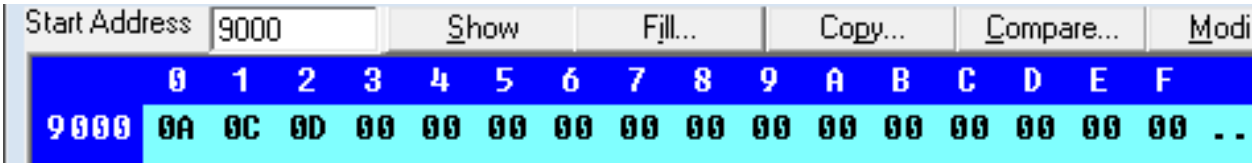


Figure 16: Experiment 01(C): Output screenshot 4.

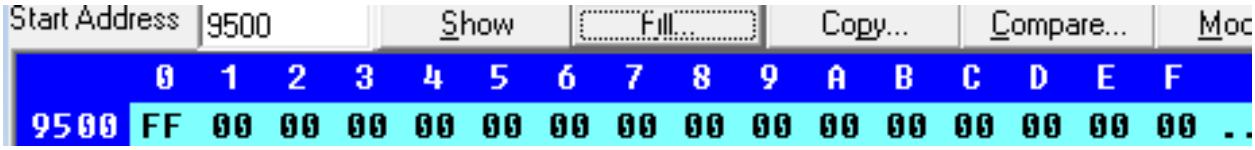


Figure 17: Experiment 01(C): Output screenshot 5.

6 Discussion

Soumadipta Majumder, 23EC30055

Experiment 01(A) — SORT and SEARCH: The bubble sort routine correctly sorts the 10-byte array stored in external RAM (9000H–9009H) by repeatedly comparing adjacent elements and swapping them when required. After sorting, the binary search routine efficiently locates the required key by halving the search interval using `low`, `high`, and `mid`; the program stores the index at 9550H when found, otherwise it stores FFH. This combination demonstrates the advantage of sorting before applying binary search for faster lookup.

Experiment 01(B): The polynomial/iterative accumulation program reads `n`, `x`, and the coefficients from external memory (starting at 9500H). It performs repeated multiply-and-add steps (Horner-like evaluation) while managing carries to produce a 16-bit result, which is finally stored

at 9550H (low byte) and 9551H (high byte). The key implementation detail here is correct propagation of carry after additions so that the high byte remains consistent.

Experiment 01(C): The substring search routine compares characters from the substring and the main string stored in external RAM pages (90xxH and 92xxH). It tracks the first matched position and handles mismatches by resetting the substring index and restarting the comparison from the next possible position. If the substring terminator (00H) is reached, the stored start index is written to 9500H; if the string terminator is reached first, FFH is written to indicate "not found". This experiment highlights the importance of careful index management to avoid skipping valid matches.

Jai Raj, 23EC10031

Experiment 01(A) — SORT and SEARCH: In this part, the array is accessed in external RAM using `DPTR`, since it is the only continuous 16-bit pointer available on the 8051 for xRAM addressing (formed by `DPH:DPL`). The bubble sort works by repeatedly scanning the 10 elements (9000H–9009H), comparing adjacent bytes and swapping whenever the next element is smaller, until the array becomes ordered. After sorting, binary search reduces the search interval by updating `low` and `high` based on comparisons with the middle element, finally storing the required *index* at 9550H (or FFH if the key is absent). A key low-level point is that all reads/writes to external memory use `MOVX` and one operand must be the accumulator (A).

Experiment 01(B): The polynomial evaluation uses an iterative multiply-and-add procedure (Horner-style accumulation) to build the result step-by-step from coefficients in xRAM. Since 8051 ALU instructions require the accumulator as one operand, intermediate arithmetic (addition, carry propagation, multiplication set-up, etc.) is performed through A, while the partial 16-bit value is maintained across registers and updated every iteration. Correct carry handling is essential so that the high byte remains consistent when the low byte overflows.

Experiment 01(C): The substring search compares characters from the substring (90xxH page) and the string (92xxH page) and keeps track of a tentative match start index. On a mismatch, the routine resets the substring index and restarts from the next possible string position so that potential matches are not skipped. One important edge case we addressed was when the main string is `aaabcd` and the substring is `aab`: our initial logic failed because we did not reset the string pointer back to the position where the comparison first started, which caused the next valid match attempt to be skipped. The program terminates when it reaches a NULL (00H) in the substring (match complete) or in the string (no match), writing the resulting start index to 9500H (or FFH). While iterating through xRAM, `INC DPTR` is used for forward traversal; however, since `DEC DPTR` is not supported directly on 8051, any backward step must be handled by decrementing `DPL/DPH` appropriately.